

Single-layer NN/Shallow NN

Regression and Classification

Introduction to Deep Learning · Class 6 · Classification

4 August

Outline

From Regression to Classification

Linear Classifiers and Decision Boundaries

The Maximum-Likelihood Principle

Binary Classification

Multi-class Classification

The Bigger Picture

From Regression to Classification

Notation used throughout this course

$\mathbf{x} \in \mathbb{R}^D$	input / features	$\phi_j(\mathbf{x})$	fixed basis function
$y, \mathbf{y}$	target , targets	a, a_k	output pre-activation
$\hat{y}(\mathbf{x}, \mathbf{w})$	model prediction	$z_j = h(a_j)$	hidden unit
$\mathcal{C}_1, \dots, \mathcal{C}_K$	the classes	$\sigma(a)$	logistic sigmoid
N, D, M, K	examples, inputs, units, classes	$E(\mathbf{w})$	error function
$\mathbf{w}, w_0$	weights, bias	δ_{kj}	Kronecker delta

Symbols follow **Bishop & Bishop (2024)**, except that targets are written y and predictions $\hat{y}$. For K classes the target is **one-hot**: $y_{nk} \in \{0, 1\}$ with $\sum_k y_{nk} = 1$.

What changes, and what does not

	Regression (Class 5)	Classification (today)
Target	$y \in \mathbb{R}$	$y \in \{0, 1\}$, or one of K classes
Output activation	identity	sigmoid / softmax
Error	sum of squares	cross-entropy
Prediction	a value	a probability

The hidden layer is **unchanged**: same architecture, same learned features $z_j = h(a_j)$. Only the last step differs.

ML linear regression $\rightarrow$ logistic regression

NN NN regression $\rightarrow$ NN classification, same translation

Recap: the shallow network

From Class 5 (Bishop & Bishop, 2024, Eqs. 6.1–6.3):

$$a_j = \sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)}, \quad z_j = h(a_j), \quad a_k = \sum_{j=1}^M w_{kj}^{(2)} z_j + w_{k0}^{(2)}$$

- Regression: $\hat{y}_k = a_k$ — the output *is* the pre-activation
- Classification: pass a_k through an **output activation** that turns it into a probability
- Everything before that last step is identical

ML $\sigma(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}))$: fixed features

NN $\sigma(\sum_j w_j^{(2)} z_j + w_0^{(2)})$: learned features

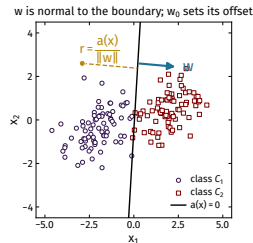
Linear Classifiers and Decision Boundaries

Discriminant functions

The simplest classifier (B&B, 2024, §5.1.1):

$$a(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0$$

- Assign $\mathcal{C}_2$ if $a(\mathbf{x}) \geq 0$, else $\mathcal{C}_1$
- The boundary $a(\mathbf{x}) = 0$ is a **hyperplane**
- $\mathbf{w}$ fixes its **orientation**, w_0 its **offset**



Two Gaussian classes and the fitted boundary.

ML a hyperplane in input space

NN a hyperplane in *learned feature space*

How far is a point from the boundary?

Theorem 1 (Signed distance to a hyperplane)

For the hyperplane $a(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0 = 0$, the signed perpendicular distance from any point $\mathbf{x}$ is

$$r = \frac{a(\mathbf{x})}{\|\mathbf{w}\|} = \frac{\mathbf{w}^\top \mathbf{x} + w_0}{\|\mathbf{w}\|}. \quad (\text{B\&B 5.7})$$

- Write $\mathbf{x} = \mathbf{x}_\perp + r\mathbf{w}/\|\mathbf{w}\|$, with $\mathbf{x}_\perp$ the foot of the perpendicular
- Apply $a(\cdot)$: $a(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}_\perp + w_0 + r\mathbf{w}^\top \mathbf{w}/\|\mathbf{w}\|$
- The first two terms are $a(\mathbf{x}_\perp) = 0$, because $\mathbf{x}_\perp$ lies on the boundary
- So $a(\mathbf{x}) = r\|\mathbf{w}\|$, that is $r = a(\mathbf{x})/\|\mathbf{w}\|$

□

$|a(\mathbf{x})|$ is a **confidence**: large means far from the boundary. The sigmoid turns exactly this quantity into a probability.

How far is a point from the boundary?

Theorem 1 (Signed distance to a hyperplane)

For the hyperplane $a(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0 = 0$, the signed perpendicular distance from any point $\mathbf{x}$ is

$$r = \frac{a(\mathbf{x})}{\|\mathbf{w}\|} = \frac{\mathbf{w}^T \mathbf{x} + w_0}{\|\mathbf{w}\|}. \quad (\text{B\&B 5.7})$$

- Write $\mathbf{x} = \mathbf{x}_\perp + r\mathbf{w}/\|\mathbf{w}\|$, with $\mathbf{x}_\perp$ the foot of the perpendicular
- **Apply $a(\cdot)$:** $a(\mathbf{x}) = \mathbf{w}^T \mathbf{x}_\perp + w_0 + r\mathbf{w}^T \mathbf{w}/\|\mathbf{w}\|$
- The first two terms are $a(\mathbf{x}_\perp) = 0$, because $\mathbf{x}_\perp$ lies on the boundary
- So $a(\mathbf{x}) = r\|\mathbf{w}\|$, that is $r = a(\mathbf{x})/\|\mathbf{w}\|$

□

$|a(\mathbf{x})|$ is a **confidence**: large means far from the boundary. The sigmoid turns exactly this quantity into a probability.

How far is a point from the boundary?

Theorem 1 (Signed distance to a hyperplane)

For the hyperplane $a(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0 = 0$, the signed perpendicular distance from any point $\mathbf{x}$ is

$$r = \frac{a(\mathbf{x})}{\|\mathbf{w}\|} = \frac{\mathbf{w}^\top \mathbf{x} + w_0}{\|\mathbf{w}\|}. \quad (\text{B\&B 5.7})$$

- Write $\mathbf{x} = \mathbf{x}_\perp + r\mathbf{w}/\|\mathbf{w}\|$, with $\mathbf{x}_\perp$ the foot of the perpendicular
- Apply $a(\cdot)$: $a(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}_\perp + w_0 + r\mathbf{w}^\top \mathbf{w}/\|\mathbf{w}\|$
- **The first two terms are $a(\mathbf{x}_\perp) = 0$, because $\mathbf{x}_\perp$ lies on the boundary**
- So $a(\mathbf{x}) = r\|\mathbf{w}\|$, that is $r = a(\mathbf{x})/\|\mathbf{w}\|$

□

$|a(\mathbf{x})|$ is a **confidence**: large means far from the boundary. The sigmoid turns exactly this quantity into a probability.

How far is a point from the boundary?

Theorem 1 (Signed distance to a hyperplane)

For the hyperplane $a(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0 = 0$, the signed perpendicular distance from any point $\mathbf{x}$ is

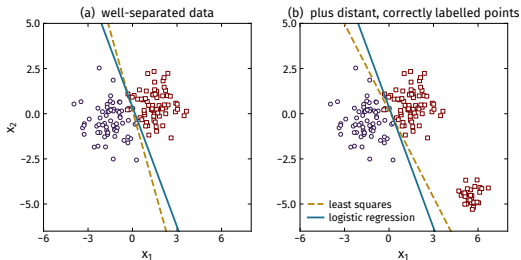
$$r = \frac{a(\mathbf{x})}{\|\mathbf{w}\|} = \frac{\mathbf{w}^\top \mathbf{x} + w_0}{\|\mathbf{w}\|}. \quad (\text{B\&B 5.7})$$

- Write $\mathbf{x} = \mathbf{x}_\perp + r\mathbf{w}/\|\mathbf{w}\|$, with $\mathbf{x}_\perp$ the foot of the perpendicular
- Apply $a(\cdot)$: $a(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}_\perp + w_0 + r\mathbf{w}^\top \mathbf{w}/\|\mathbf{w}\|$
- The first two terms are $a(\mathbf{x}_\perp) = 0$, because $\mathbf{x}_\perp$ lies on the boundary
- **So $a(\mathbf{x}) = r\|\mathbf{w}\|$, that is $r = a(\mathbf{x})/\|\mathbf{w}\|$**

□

$|a(\mathbf{x})|$ is a **confidence**: large means far from the boundary. The sigmoid turns exactly this quantity into a probability.

Why not just do least squares?



The same two classes, fitted twice. Dashed amber: least squares on 0/1 targets. Solid turquoise: logistic regression.

- Distant but *correctly classified* points drag the least-squares boundary — they are penalised for being “too right”
- Squared error is the wrong error for targets in $\{0, 1\}$; we need one built for probabilities

Where the sigmoid comes from

Theorem 2 (Gaussian classes give a logistic posterior)

Let $p(\mathbf{x} | \mathcal{C}_k) = \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma})$ with a *shared* covariance $\boldsymbol{\Sigma}$. Then

$$p(\mathcal{C}_2 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + w_0), \quad \mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1).$$

- By Bayes, $p(\mathcal{C}_2 | \mathbf{x}) = \sigma(a)$ where a is the log-odds $a = \ln \frac{p(\mathbf{x} | \mathcal{C}_2)p(\mathcal{C}_2)}{p(\mathbf{x} | \mathcal{C}_1)p(\mathcal{C}_1)}$
- Substitute the two Gaussians. The quadratic terms $-\frac{1}{2}\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}$ are *identical* and cancel — this is exactly where the shared $\boldsymbol{\Sigma}$ is used
- What survives is linear in $\mathbf{x}$: $a = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \boldsymbol{\Sigma}^{-1}\mathbf{x} + w_0$ □
- Drop the shared-covariance assumption and the quadratic terms remain: the boundary becomes a *quadratic* surface

Where the sigmoid comes from

Theorem 2 (Gaussian classes give a logistic posterior)

Let $p(\mathbf{x} | \mathcal{C}_k) = \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma})$ with a *shared* covariance $\boldsymbol{\Sigma}$. Then

$$p(\mathcal{C}_2 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + w_0), \quad \mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1).$$

- By Bayes, $p(\mathcal{C}_2 | \mathbf{x}) = \sigma(a)$ where a is the **log-odds** $a = \ln \frac{p(\mathbf{x} | \mathcal{C}_2)p(\mathcal{C}_2)}{p(\mathbf{x} | \mathcal{C}_1)p(\mathcal{C}_1)}$
- **Substitute the two Gaussians. The quadratic terms $-\frac{1}{2}\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}$ are *identical* and cancel — this is exactly where the shared $\boldsymbol{\Sigma}$ is used**
- What survives is linear in $\mathbf{x}$: $a = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \boldsymbol{\Sigma}^{-1}\mathbf{x} + w_0$ □
- Drop the shared-covariance assumption and the quadratic terms remain: the boundary becomes a **quadratic** surface

Where the sigmoid comes from

Theorem 2 (Gaussian classes give a logistic posterior)

Let $p(\mathbf{x} | \mathcal{C}_k) = \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma})$ with a *shared* covariance $\boldsymbol{\Sigma}$. Then

$$p(\mathcal{C}_2 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + w_0), \quad \mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1).$$

- By Bayes, $p(\mathcal{C}_2 | \mathbf{x}) = \sigma(a)$ where a is the **log-odds** $a = \ln \frac{p(\mathbf{x} | \mathcal{C}_2)p(\mathcal{C}_2)}{p(\mathbf{x} | \mathcal{C}_1)p(\mathcal{C}_1)}$
- Substitute the two Gaussians. The quadratic terms $-\frac{1}{2}\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}$ are *identical* and cancel — this is exactly where the shared $\boldsymbol{\Sigma}$ is used
- **What survives is linear in $\mathbf{x}$:** $a = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \boldsymbol{\Sigma}^{-1}\mathbf{x} + w_0$ □
- Drop the shared-covariance assumption and the quadratic terms remain: the boundary becomes a **quadratic** surface

Where the sigmoid comes from

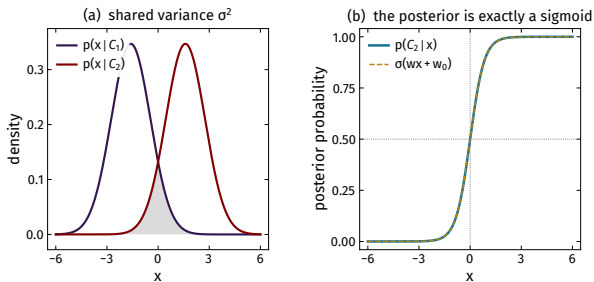
Theorem 2 (Gaussian classes give a logistic posterior)

Let $p(\mathbf{x} \mid \mathcal{C}_k) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma})$ with a *shared* covariance $\boldsymbol{\Sigma}$. Then

$$p(\mathcal{C}_2 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + w_0), \quad \mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1).$$

- By Bayes, $p(\mathcal{C}_2 \mid \mathbf{x}) = \sigma(a)$ where a is the **log-odds** $a = \ln \frac{p(\mathbf{x} \mid \mathcal{C}_2)p(\mathcal{C}_2)}{p(\mathbf{x} \mid \mathcal{C}_1)p(\mathcal{C}_1)}$
- Substitute the two Gaussians. The quadratic terms $-\frac{1}{2}\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}$ are *identical* and cancel — this is exactly where the shared $\boldsymbol{\Sigma}$ is used
- What survives is linear in $\mathbf{x}$: $a = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \boldsymbol{\Sigma}^{-1}\mathbf{x} + w_0$ □
- **Drop the shared-covariance assumption and the quadratic terms remain: the boundary becomes a quadratic surface**

The sigmoid, seen in the data



(a) two Gaussian class-conditionals sharing one variance; shaded is their overlap. (b) the exact posterior $p(C_2 | x)$ in turquoise, with $\sigma(wx + w_0)$ from the theorem dashed on top — the two coincide.

The sigmoid is the **natural form of a posterior probability**, not an arbitrary squashing choice.

The Maximum-Likelihood Principle

Where the error function comes from

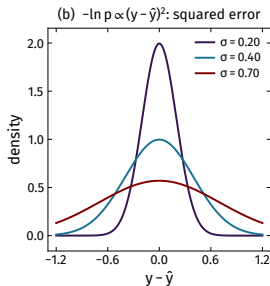
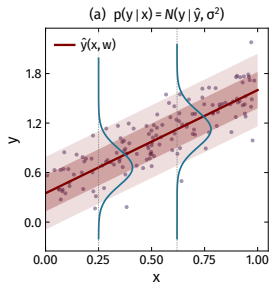
Rather than choosing an error function by hand, we **derive** it. The network predicts the parameters of a probability distribution over the output:

1. Choose a distribution $p(y \mid \cdot)$ suited to the **domain of y**
2. Let the network predict its parameters
3. Train by minimising the **negative log-likelihood**

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \left[- \sum_{n=1}^N \ln p(y_n \mid \hat{y}(\mathbf{x}_n, \mathbf{w})) \right]$$

- The logarithm turns products of small numbers into sums, and does not move the maximum
- Inference: report the **peak** of the predicted distribution

You already know one instance



Class 5, restated in this framework:

- Domain $y \in \mathbb{R} \rightarrow$ choose a **Gaussian**
- The network predicts its mean $\hat{y}$
- $-\ln p = \frac{(y - \hat{y})^2}{2\sigma^2} + \text{const} \rightarrow$ **squared error**

(a) the model predicts the *mean* of a Gaussian at each x . (b)
its negative log density is a parabola in $y - \hat{y}$.

ML Gaussian noise $\rightarrow$ squared error

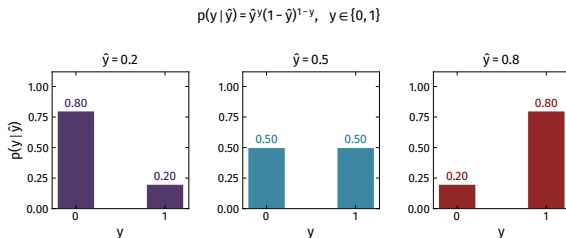
NN today: Bernoulli $\rightarrow$ cross-entropy, same construction

Binary Classification

Step 1: the Bernoulli distribution

Domain $y \in \{0, 1\}$; one parameter, taken to be the prediction:

$$p(y | \hat{y}) = \hat{y}^y (1 - \hat{y})^{1-y}, \quad \hat{y} \in [0, 1]$$



The whole distribution is one number — so the network must output a value in $[0, 1]$, while the pre-activation a is an arbitrary real number.

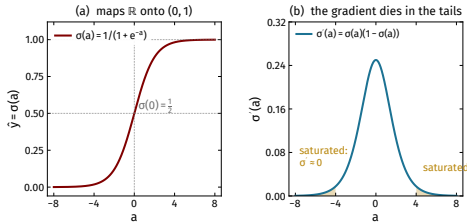
Step 2: the logistic sigmoid

Map $\mathbb{R}$ onto $(0, 1)$ (B&B, 2024, Eq. 5.42):

$$\hat{y} = \sigma(a) = \frac{1}{1 + \exp(-a)}$$

- a is the **log-odds**: $a = \ln \frac{\hat{y}}{1 - \hat{y}}$
- $a/\|\mathbf{w}\|$ was the distance to the boundary — distance becomes confidence

ML $\sigma(\mathbf{w}^\top \phi(\mathbf{x}))$



(a) the sigmoid. (b) its derivative, which all but vanishes
once $|a| \gtrsim 4$.

NN the same sigmoid, on a learned-feature read-out

Properties we will need

Proposition 3 (The logistic sigmoid)

For all $a \in \mathbb{R}$:

$$(i) \sigma(-a) = 1 - \sigma(a), \quad (ii) \sigma'(a) = \sigma(a)(1 - \sigma(a)), \quad (iii) \sigma^{-1}(u) = \ln \frac{u}{1-u}.$$

- (i) $\sigma(-a) = \frac{1}{1+e^a} = \frac{e^{-a}}{e^{-a}+1} = 1 - \frac{1}{1+e^{-a}}$
- (ii) Differentiate: $\sigma'(a) = \frac{e^{-a}}{(1+e^{-a})^2} = \frac{1}{1+e^{-a}} \cdot \frac{e^{-a}}{1+e^{-a}} = \sigma(a)(1 - \sigma(a))$
- (iii) Solve $u = 1/(1+e^{-a})$: $e^{-a} = (1-u)/u$, hence $a = \ln \frac{u}{1-u}$ □

(ii) is the identity that makes the cross-entropy gradient collapse to something very simple; (iii) says the pre-activation is the log-odds.

Properties we will need

Proposition 3 (The logistic sigmoid)

For all $a \in \mathbb{R}$:

$$(i) \sigma(-a) = 1 - \sigma(a), \quad (ii) \sigma'(a) = \sigma(a)(1 - \sigma(a)), \quad (iii) \sigma^{-1}(u) = \ln \frac{u}{1-u}.$$

- (i) $\sigma(-a) = \frac{1}{1+e^a} = \frac{e^{-a}}{e^{-a}+1} = 1 - \frac{1}{1+e^{-a}}$
- (ii) Differentiate: $\sigma'(a) = \frac{e^{-a}}{(1+e^{-a})^2} = \frac{1}{1+e^{-a}} \cdot \frac{e^{-a}}{1+e^{-a}} = \sigma(a)(1 - \sigma(a))$
- (iii) Solve $u = 1/(1+e^{-a})$: $e^{-a} = (1-u)/u$, hence $a = \ln \frac{u}{1-u}$

□

(ii) is the identity that makes the cross-entropy gradient collapse to something very simple; (iii) says the pre-activation is the log-odds.

Properties we will need

Proposition 3 (The logistic sigmoid)

For all $a \in \mathbb{R}$:

$$(i) \sigma(-a) = 1 - \sigma(a), \quad (ii) \sigma'(a) = \sigma(a)(1 - \sigma(a)), \quad (iii) \sigma^{-1}(u) = \ln \frac{u}{1-u}.$$

- (i) $\sigma(-a) = \frac{1}{1+e^a} = \frac{e^{-a}}{e^{-a}+1} = 1 - \frac{1}{1+e^{-a}}$
- (ii) Differentiate: $\sigma'(a) = \frac{e^{-a}}{(1+e^{-a})^2} = \frac{1}{1+e^{-a}} \cdot \frac{e^{-a}}{1+e^{-a}} = \sigma(a)(1 - \sigma(a))$
- (iii) Solve $u = 1/(1+e^{-a})$: $e^{-a} = (1-u)/u$, hence $a = \ln \frac{u}{1-u}$ □

(ii) is the identity that makes the cross-entropy gradient collapse to something very simple; (iii) says the pre-activation is the log-odds.

Step 3: binary cross-entropy

Negative log-likelihood of the Bernoulli model over N independent examples (B&B, 2024, Eq. 5.74):

$$E(\mathbf{w}) = - \sum_{n=1}^N \left[y_n \ln \hat{y}_n + (1 - y_n) \ln(1 - \hat{y}_n) \right], \quad \hat{y}_n = \sigma(a_n)$$

- Only one term is active per example: $y_n=1$ charges $-\ln \hat{y}_n$, and $y_n=0$ charges $-\ln(1 - \hat{y}_n)$
- Both are zero for a perfect prediction and diverge for a confident mistake

ML logistic regression: convex in $\mathbf{w}$

NN same error, non-convex once a network is inside

The gradient is as simple as it could be

Theorem 4 (Cross-entropy gradient)

With $\hat{y}_n = \sigma(a_n)$ and E the binary cross-entropy,

$$\frac{\partial E}{\partial a_n} = \hat{y}_n - y_n, \quad \text{so} \quad \nabla_{\mathbf{w}} E = \sum_{n=1}^N (\hat{y}_n - y_n) \phi_n \quad (\text{B\&B 5.75})$$

for a linear model $a_n = \mathbf{w}^\top \phi_n$.

- $\frac{\partial E}{\partial \hat{y}_n} = -\frac{y_n}{\hat{y}_n} + \frac{1-y_n}{1-\hat{y}_n} = \frac{\hat{y}_n - y_n}{\hat{y}_n(1-\hat{y}_n)}$
- By Proposition (ii), $\frac{\partial \hat{y}_n}{\partial a_n} = \hat{y}_n(1-\hat{y}_n)$
- Chain rule: the two factors $\hat{y}_n(1-\hat{y}_n)$ **cancel exactly**, leaving $\partial E / \partial a_n = \hat{y}_n - y_n$
- Finally $\partial a_n / \partial \mathbf{w} = \phi_n$ gives the sum

□

The gradient is as simple as it could be

Theorem 4 (Cross-entropy gradient)

With $\hat{y}_n = \sigma(a_n)$ and E the binary cross-entropy,

$$\frac{\partial E}{\partial a_n} = \hat{y}_n - y_n, \quad \text{so} \quad \nabla_{\mathbf{w}} E = \sum_{n=1}^N (\hat{y}_n - y_n) \phi_n \quad (\text{B\&B 5.75})$$

for a linear model $a_n = \mathbf{w}^\top \phi_n$.

- $\frac{\partial E}{\partial \hat{y}_n} = -\frac{y_n}{\hat{y}_n} + \frac{1-y_n}{1-\hat{y}_n} = \frac{\hat{y}_n - y_n}{\hat{y}_n(1-\hat{y}_n)}$
- By Proposition (ii), $\frac{\partial \hat{y}_n}{\partial a_n} = \hat{y}_n(1-\hat{y}_n)$
- Chain rule: the two factors $\hat{y}_n(1-\hat{y}_n)$ cancel exactly, leaving $\partial E / \partial a_n = \hat{y}_n - y_n$
- Finally $\partial a_n / \partial \mathbf{w} = \phi_n$ gives the sum

□

The gradient is as simple as it could be

Theorem 4 (Cross-entropy gradient)

With $\hat{y}_n = \sigma(a_n)$ and E the binary cross-entropy,

$$\frac{\partial E}{\partial a_n} = \hat{y}_n - y_n, \quad \text{so} \quad \nabla_{\mathbf{w}} E = \sum_{n=1}^N (\hat{y}_n - y_n) \phi_n \quad (\text{B\&B 5.75})$$

for a linear model $a_n = \mathbf{w}^\top \phi_n$.

- $\frac{\partial E}{\partial \hat{y}_n} = -\frac{y_n}{\hat{y}_n} + \frac{1-y_n}{1-\hat{y}_n} = \frac{\hat{y}_n - y_n}{\hat{y}_n(1-\hat{y}_n)}$
- By Proposition (ii), $\frac{\partial \hat{y}_n}{\partial a_n} = \hat{y}_n(1-\hat{y}_n)$
- Chain rule: the two factors $\hat{y}_n(1-\hat{y}_n)$ cancel exactly, leaving $\partial E / \partial a_n = \hat{y}_n - y_n$**
- Finally $\partial a_n / \partial \mathbf{w} = \phi_n$ gives the sum



The gradient is as simple as it could be

Theorem 4 (Cross-entropy gradient)

With $\hat{y}_n = \sigma(a_n)$ and E the binary cross-entropy,

$$\frac{\partial E}{\partial a_n} = \hat{y}_n - y_n, \quad \text{so} \quad \nabla_{\mathbf{w}} E = \sum_{n=1}^N (\hat{y}_n - y_n) \phi_n \quad (\text{B\&B 5.75})$$

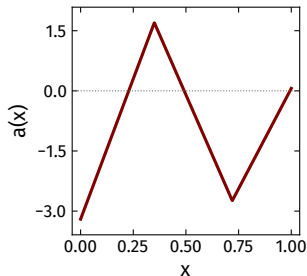
for a linear model $a_n = \mathbf{w}^\top \phi_n$.

- $\frac{\partial E}{\partial \hat{y}_n} = -\frac{y_n}{\hat{y}_n} + \frac{1-y_n}{1-\hat{y}_n} = \frac{\hat{y}_n - y_n}{\hat{y}_n(1-\hat{y}_n)}$
- By Proposition (ii), $\frac{\partial \hat{y}_n}{\partial a_n} = \hat{y}_n(1-\hat{y}_n)$
- Chain rule: the two factors $\hat{y}_n(1-\hat{y}_n)$ **cancel exactly**, leaving $\partial E / \partial a_n = \hat{y}_n - y_n$
- Finally $\partial a_n / \partial \mathbf{w} = \phi_n$ **gives the sum**

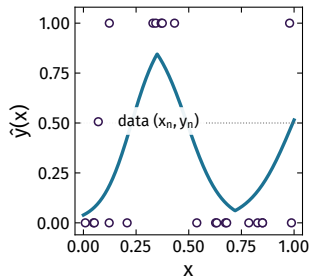
□

The binary model, end to end

(a) network output $a(x)$: piecewise linear

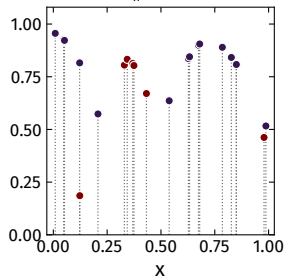


(b) $\hat{y} = \sigma(a)$: a probability



likelihood of the observed label

(c) $\hat{y}_n^{y_n}(1 - \hat{y}_n)^{1 - y_n}$

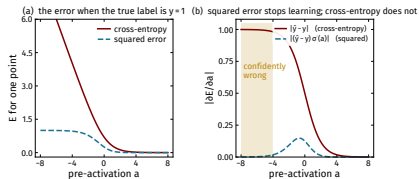


(a) the network output $a(x)$ — still piecewise linear, exactly as in Class 5. (b) after the sigmoid it is a probability, with the observed labels overlaid. (c) the likelihood each point assigns to the label it actually carries; training pushes all of these upwards.

Why not squared error on probabilities?

Proposition 5 (Squared error saturates)

With a sigmoid output, $\frac{\partial E_{\text{SSE}}}{\partial a} = (\hat{y} - y) \sigma'(a)$, whereas $\frac{\partial E_{\text{CE}}}{\partial a} = \hat{y} - y$. Since $\sigma'(a) \rightarrow 0$ as $|a| \rightarrow \infty$, the squared-error gradient vanishes even when the prediction is confidently wrong.



True label $y = 1$. Shaded: confidently wrong. The squared-error gradient dies there; cross-entropy stays at full strength.

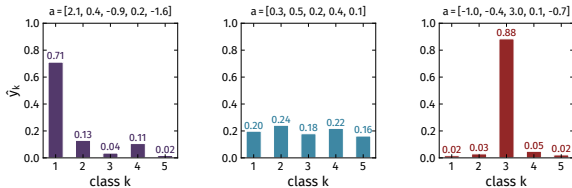
Multi-class Classification

Step 1: the categorical distribution

Domain $y \in \{1, \dots, K\}$, one-hot encoded; K parameters:

$$p(y = k) = \hat{y}_k, \quad \hat{y}_k \geq 0, \quad \sum_{k=1}^K \hat{y}_k = 1$$

$$\hat{y}_k = \frac{\exp(a_k)}{\sum_j \exp(a_j)}: \text{non-negative, sums to one}$$



Three sets of pre-activations and the distributions they produce; the raw $a_1, \dots, a_K$ are unconstrained.

Step 2: softmax

Normalise the exponentiated pre-activations (B&B, 2024, Eqs. 5.76–5.77):

$$\hat{y}_k = \text{softmax}_k(\mathbf{a}) = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)}$$

Proposition 6 (Softmax)

(i) $\hat{y}_k > 0$ and $\sum_k \hat{y}_k = 1$. (ii) *Shift invariance*: $\text{softmax}(\mathbf{a} + c\mathbf{1}) = \text{softmax}(\mathbf{a})$ for any c . (iii) For $K = 2$ it *is* the sigmoid: $\hat{y}_1 = \sigma(a_1 - a_2)$.

Proof. (i) is immediate. (ii) multiplies numerator and denominator by e^c . (iii) $\frac{e^{a_1}}{e^{a_1} + e^{a_2}} = \frac{1}{1 + e^{-(a_1 - a_2)}}$. □

(ii) is what makes the computation numerically safe: subtract $\max_k a_k$ before exponentiating.

ML multinomial logistic regression

NN softmax on K learned-feature read-outs

The softmax Jacobian

Lemma 7 (Derivative of softmax)

For $\hat{y}_k = \text{softmax}_k(\mathbf{a})$,

$$\frac{\partial \hat{y}_k}{\partial a_j} = \hat{y}_k (\delta_{kj} - \hat{y}_j), \quad \delta_{kj} = \begin{cases} 1 & k = j \\ 0 & k \neq j. \end{cases}$$

- Write $\hat{y}_k = e^{a_k}/S$ with $S = \sum_j e^{a_j}$, so $\partial S/\partial a_j = e^{a_j}$
- Quotient rule: $\frac{\partial \hat{y}_k}{\partial a_j} = \frac{\delta_{kj} e^{a_k} S - e^{a_k} e^{a_j}}{S^2}$
- Divide through and recognise $e^{a_k}/S = \hat{y}_k$, $e^{a_j}/S = \hat{y}_j$: $= \hat{y}_k \delta_{kj} - \hat{y}_k \hat{y}_j$ □
- Check $K = 2$: this gives $\hat{y}_1(1 - \hat{y}_1)$ — the sigmoid derivative again

The softmax Jacobian

Lemma 7 (Derivative of softmax)

For $\hat{y}_k = \text{softmax}_k(\mathbf{a})$,

$$\frac{\partial \hat{y}_k}{\partial a_j} = \hat{y}_k (\delta_{kj} - \hat{y}_j), \quad \delta_{kj} = \begin{cases} 1 & k = j \\ 0 & k \neq j. \end{cases}$$

- Write $\hat{y}_k = e^{a_k}/S$ with $S = \sum_j e^{a_j}$, so $\partial S/\partial a_j = e^{a_j}$
- **Quotient rule:** $\frac{\partial \hat{y}_k}{\partial a_j} = \frac{\delta_{kj} e^{a_k} S - e^{a_k} e^{a_j}}{S^2}$
- Divide through and recognise $e^{a_k}/S = \hat{y}_k$, $e^{a_j}/S = \hat{y}_j$: $= \hat{y}_k \delta_{kj} - \hat{y}_k \hat{y}_j$ □
- Check $K = 2$: this gives $\hat{y}_1(1 - \hat{y}_1)$ — the sigmoid derivative again

The softmax Jacobian

Lemma 7 (Derivative of softmax)

For $\hat{y}_k = \text{softmax}_k(\mathbf{a})$,

$$\frac{\partial \hat{y}_k}{\partial a_j} = \hat{y}_k (\delta_{kj} - \hat{y}_j), \quad \delta_{kj} = \begin{cases} 1 & k = j \\ 0 & k \neq j. \end{cases}$$

- Write $\hat{y}_k = e^{a_k}/S$ with $S = \sum_j e^{a_j}$, so $\partial S/\partial a_j = e^{a_j}$
- Quotient rule: $\frac{\partial \hat{y}_k}{\partial a_j} = \frac{\delta_{kj} e^{a_k} S - e^{a_k} e^{a_j}}{S^2}$
- **Divide through and recognise $e^{a_k}/S = \hat{y}_k$, $e^{a_j}/S = \hat{y}_j$:** $= \hat{y}_k \delta_{kj} - \hat{y}_k \hat{y}_j$ □
- Check $K = 2$: this gives $\hat{y}_1(1 - \hat{y}_1)$ — the sigmoid derivative again

The softmax Jacobian

Lemma 7 (Derivative of softmax)

For $\hat{y}_k = \text{softmax}_k(\mathbf{a})$,

$$\frac{\partial \hat{y}_k}{\partial a_j} = \hat{y}_k (\delta_{kj} - \hat{y}_j), \quad \delta_{kj} = \begin{cases} 1 & k = j \\ 0 & k \neq j. \end{cases}$$

- Write $\hat{y}_k = e^{a_k}/S$ with $S = \sum_j e^{a_j}$, so $\partial S/\partial a_j = e^{a_j}$
- Quotient rule: $\frac{\partial \hat{y}_k}{\partial a_j} = \frac{\delta_{kj} e^{a_k} S - e^{a_k} e^{a_j}}{S^2}$
- Divide through and recognise $e^{a_k}/S = \hat{y}_k$, $e^{a_j}/S = \hat{y}_j$: $= \hat{y}_k \delta_{kj} - \hat{y}_k \hat{y}_j$ □
- Check $K = 2$: this gives $\hat{y}_1(1 - \hat{y}_1)$ — the sigmoid derivative again

Step 3: multi-class cross-entropy

Theorem 8 (Cross-entropy and its gradient)

With one-hot targets y_{nk} and $\hat{y}_{nk} = \text{softmax}_k(\mathbf{a}_n)$,

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K y_{nk} \ln \hat{y}_{nk}, \quad \frac{\partial E}{\partial a_{nk}} = \hat{y}_{nk} - y_{nk}. \quad (\text{B\&B 5.80--5.81})$$

- $\frac{\partial E_n}{\partial a_{nk}} = - \sum_c y_{nc} \frac{\partial \hat{y}_{nc}}{\partial a_{nk}}$
- Insert the Lemma: $= - \sum_c \frac{y_{nc}}{\hat{y}_{nc}} \hat{y}_{nc} (\delta_{ck} - \hat{y}_{nk}) = - \sum_c y_{nc} (\delta_{ck} - \hat{y}_{nk})$
- Split: $= -y_{nk} + \hat{y}_{nk} \sum_c y_{nc} = \hat{y}_{nk} - y_{nk}$, since $\sum_c y_{nc} = 1$ □

Regression, binary, multi-class: the output-layer gradient is $\hat{y} - y$ every time.

Step 3: multi-class cross-entropy

Theorem 8 (Cross-entropy and its gradient)

With one-hot targets y_{nk} and $\hat{y}_{nk} = \text{softmax}_k(\mathbf{a}_n)$,

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K y_{nk} \ln \hat{y}_{nk}, \quad \frac{\partial E}{\partial a_{nk}} = \hat{y}_{nk} - y_{nk}. \quad (\text{B\&B 5.80--5.81})$$

- $\frac{\partial E_n}{\partial a_{nk}} = - \sum_c y_{nc} \frac{\partial \hat{y}_{nc}}{\partial a_{nk}}$
- **Insert the Lemma:** $= - \sum_c y_{nc} \frac{\partial \hat{y}_{nc}}{\partial a_{nk}} \hat{y}_{nc} (\delta_{ck} - \hat{y}_{nk}) = - \sum_c y_{nc} (\delta_{ck} - \hat{y}_{nk})$
- **Split:** $= -y_{nk} + \hat{y}_{nk} \sum_c y_{nc} = \hat{y}_{nk} - y_{nk}$, since $\sum_c y_{nc} = 1$ □

Regression, binary, multi-class: the output-layer gradient is $\hat{y} - y$ every time.

Step 3: multi-class cross-entropy

Theorem 8 (Cross-entropy and its gradient)

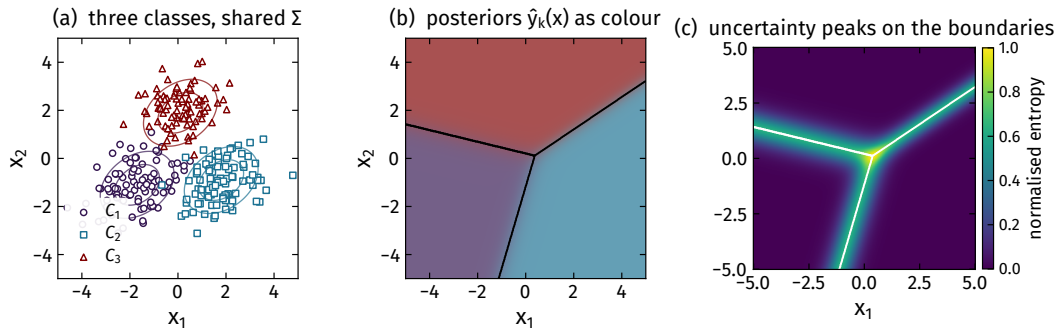
With one-hot targets y_{nk} and $\hat{y}_{nk} = \text{softmax}_k(\mathbf{a}_n)$,

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K y_{nk} \ln \hat{y}_{nk}, \quad \frac{\partial E}{\partial a_{nk}} = \hat{y}_{nk} - y_{nk}. \quad (\text{B\&B 5.80--5.81})$$

- $\frac{\partial E_n}{\partial a_{nk}} = - \sum_c y_{nc} \frac{\partial \hat{y}_{nc}}{\partial a_{nk}}$
- Insert the Lemma: $= - \sum_c \frac{y_{nc}}{\hat{y}_{nc}} \hat{y}_{nc} (\delta_{ck} - \hat{y}_{nk}) = - \sum_c y_{nc} (\delta_{ck} - \hat{y}_{nk})$
- **Split:** $= -y_{nk} + \hat{y}_{nk} \sum_c y_{nc} = \hat{y}_{nk} - y_{nk}$, since $\sum_c y_{nc} = 1$ □

Regression, binary, multi-class: the output-layer gradient is $\hat{y} - y$ every time.

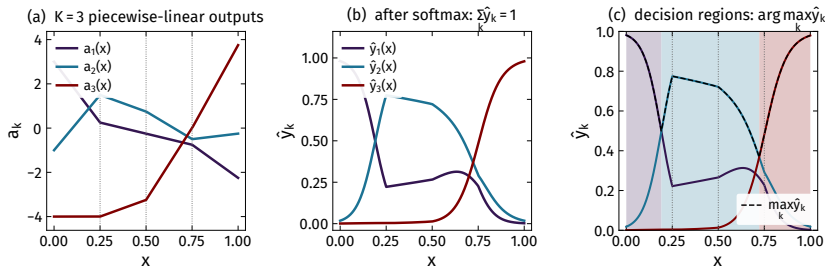
Softmax posteriors over the input space



(a) three Gaussian classes with a shared covariance. (b) the posteriors as a colour blend, decision boundaries in black — each region is where one class dominates. (c) the entropy of the posterior: the model is uncertain exactly along the boundaries, and most uncertain where all three meet.

The multi-class model on a scalar input

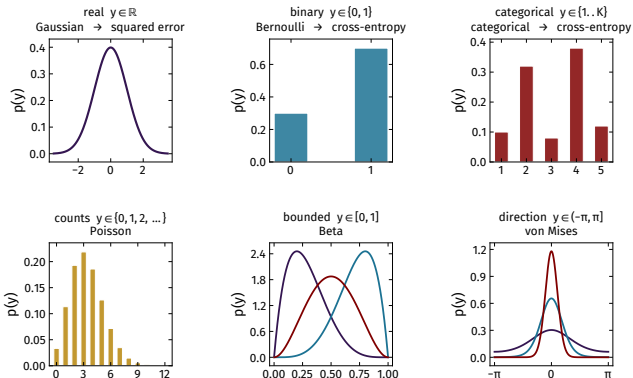
shared hidden units $\Rightarrow$ all three outputs bend at the same x



(a) $K = 3$ pre-activations from one shared hidden layer, so they bend at the same x (dotted). (b) after softmax they are non-negative and sum to one. (c) the decision regions $\arg \max_k \hat{y}_k$, shaded; the dashed line is the winning probability, which dips where the regions meet.

The Bigger Picture

One principle, any output type



Match the distribution to the domain of y and the error function follows. Counts, bounded values and directions each have their own natural choice.

Cross-entropy: the second view

Theorem 9 (Maximum likelihood = minimum KL divergence)

Let q be the empirical distribution of the data and p the model. Then

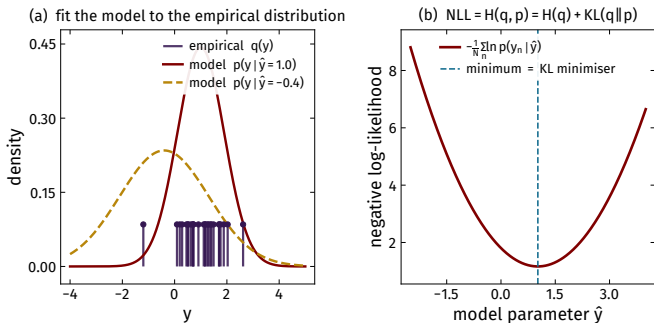
$$-\frac{1}{N} \sum_n \ln p(y_n) = H(q, p) = \underbrace{H(q)}_{\text{fixed by the data}} + \text{KL}(q \parallel p),$$

so minimising the negative log-likelihood is exactly minimising $\text{KL}(q \parallel p)$.

Proof. $H(q, p) = -\sum_y q(y) \ln p(y)$; with q the empirical distribution this average is the mean negative log-likelihood. Add and subtract $\sum_y q \ln q$ to get $H(q, p) = H(q) + \sum_y q \ln(q/p)$. The first term does not involve the model, and $\text{KL} \geq 0$ with equality only when $p = q$. $\square$

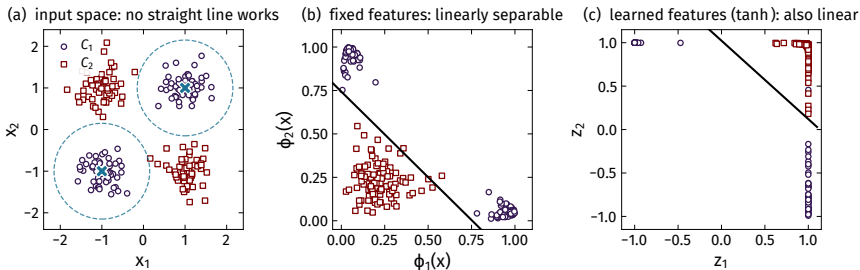
Hence the name **cross-entropy**: it is $H(q, p)$, the cost of coding data from q using the model p .

The same statement, drawn



(a) the empirical distribution (spikes at the observed values) and two candidate models; the dashed one places little mass where the data actually are. (b) the negative log-likelihood as the model parameter varies — its minimum is the point of smallest KL divergence.

Fixed features vs. learned features, again



(a) four clusters in XOR arrangement: no straight line separates them. (b) mapped by two *fixed* Gaussian basis functions centred at the crosses — now a single line does it. (c) mapped by the two *learned* $\tanh$ hidden units of a trained network: also linearly separable, but the network chose the map itself.

ML fixed ϕ , chosen to linearise the problem

NN a network *learns* a map that linearises it

Output activations: the complete table

Task	Distribution	Output activation	Error
Regression	Gaussian	identity	sum of squares
Binary	Bernoulli	sigmoid $\sigma(a)$	binary cross-entropy
Multi-class	categorical	softmax	cross-entropy

In all three cases the output-layer gradient is $\hat{y}_n - y_n$: prediction minus target.

Choose the distribution first. The output activation and the error function are then **determined**, not chosen.

The dictionary (Classes 5–6)

	Linear/Basis-Function Model	Shallow Neural Network
Features	$\phi_j(\mathbf{x})$, fixed	$z_j = h(a_j)$, learned
Regression	$\mathbf{w}^\top \phi$; squared error; normal equations	linear read-out; squared error; gradient descent
Binary	$\sigma(\mathbf{w}^\top \phi)$; cross-entropy; convex	$\sigma(\text{read-out})$; cross-entropy; non-convex
Multi-class	softmax; cross-entropy; convex	softmax; cross-entropy; non-convex
Boundary	hyperplane in a <i>chosen</i> feature space	hyperplane in a <i>learned</i> feature space

Classification is the Class-5 network with a new output activation and the error function that matches it.

Takeaways

1. The error function follows from a distribution over outputs: pick it to match the domain of y , predict its parameters, minimise the negative log-likelihood
2. Binary: Bernoulli $\rightarrow$ sigmoid $\rightarrow$ binary cross-entropy. The sigmoid is not a convenience — for Gaussian classes with shared covariance the posterior is a sigmoid, exactly
3. Multi-class: categorical $\rightarrow$ softmax $\rightarrow$ cross-entropy, and softmax reduces to the sigmoid when $K = 2$
4. Every case has output gradient $\hat{y} - y$; mismatching the error to the distribution reintroduces a σ' factor and learning stalls
5. Minimising the negative log-likelihood is minimising a KL divergence to the empirical distribution

Class 7: stacking hidden layers — deep networks, and why depth buys more than width.

References

- C. M. Bishop & H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. §5.1 (discriminant functions and their geometry), §5.2 (decision theory), §5.3 (probabilistic generative models, the logistic posterior), §5.4 (logistic regression, cross-entropy and its gradients), §6.1 (feed-forward networks).
- S. J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. Ch. 5 — deriving the error function from a distribution over outputs, and the catalogue of output distributions.
- All plots generated by `src/build_all.py` (NumPy, Matplotlib, SciencePlots); one module per figure under `src/figures/`. Notation follows `_shared/NOTATION.md`.

Questions?